

Machine Learning

Lecture 3: From Fitting to Generalization

A textbook treatment

Based on the lecture notes and slides of Mounir Ghogho

September 25, 2026

Contents

How to read this book	2
3 From Fitting to Generalization	3
How to read the formulas in this chapter	3
3.1 Generalization and Inductive Bias	3
3.2 Model Capacity	6
3.3 Bias and Variance	8
3.4 Controlling Complexity	11
3.5 Feature and Model Selection	14
3.6 Reliable Evaluation and Data Splitting	16
3.7 Diagnosis and Lab Preparation	19
Notes on the original slides and notes	20
Chapter takeaways	21

How to read this book

Each concept in this book is presented in the same rhythm.

- 1. Motivation and intuition.** Why do we care? What problem does the idea solve, and what picture should you hold in your head?
- 2. Formal statement.** The precise definition or theorem, set in a coloured box so it is easy to find when revising.
- 3. Proof.** Every claim is proved. Longer proofs are split into labelled steps, and the one idea that makes the proof work is isolated in a *Key insight* box.
- 4. Interpretation and discussion.** What the result really says, where it applies, and which misreadings to avoid.
- 5. Examples.** From “trivially small” to “genuinely useful.”
- 6. Techniques, classic problems and gotchas.** Computational shortcuts, proof patterns, exam favourites with one-line solutions, and the mistakes everyone makes once.

Margin notes carry the informal, “say it out loud” version of what is happening in the main text. Every major section opens with a *What are we trying to get?* box that states the goal in one breath before any mathematics, and closes with a summary box.

Prerequisites. Basic probability (random variables, conditional distributions, expectations), vectors in $\mathbb{R}^d$, and a first course in calculus. The information-theoretic quantities used (entropy, cross-entropy, Kullback–Leibler divergence) are defined and proved from scratch where they appear.

Chapter 3

From Fitting to Generalization

How to read the formulas in this chapter

This chapter reuses the symbols of Chapter 2 ($R, \hat{R}, \mathcal{H}, \lambda, \mathcal{D}_{\text{train}}, \mathcal{D}_{\text{val}}, \mathcal{D}_{\text{test}}$) and adds the few below. As before, every formula is followed by a *Reading the formula* box.

Symbol	Meaning in words
$\hat{f}_{\mathcal{D}}$	the model a fixed learning procedure produces when fed the dataset $\mathcal{D}$
$G(\hat{f}_{\mathcal{D}})$	the <i>generalization gap</i> : population risk minus training risk of the same fitted model
$\mathbb{E}_{\mathcal{D}}[\cdot]$	an average over <i>many possible training sets</i> drawn from the same world
$\bar{f}(x) = \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)]$	the average prediction at x across those training sets
$\text{Bias}(x), \text{Var}(x)$	systematic error and sample-to-sample spread of the prediction at x
$\varepsilon, \sigma^2(x)$	the noise in Y around the best possible prediction, and its variance
$\Omega(\theta)$	a <i>penalty</i> : a number saying how “complicated” the parameters θ are
$\ \theta\ _2^2 = \sum_j \theta_j^2, \ \theta\ _1 = \sum_j \theta_j $	the two standard penalties (squared length, and sum of absolute values)
$h \in \mathcal{C}$	one full <i>configuration</i> of hyperparameters, from a candidate set $\mathcal{C}$
$\hat{f}_h^{(-j)}$	the model with configuration h fitted on all folds <i>except</i> fold j
TP, FP, TN, FN	counts of true/false positives and negatives on a test set

3.1 Generalization and Inductive Bias

What are we trying to get?

Chapter 2 taught us to *fit*: choose the parameters that make the training examples look good. But nobody deploys a model to re-predict its own training set. We want a precise name for “does the fit carry over to new cases,” a clear reason why training performance cannot answer that question, and an honest account of what *does* decide the answer: the assumptions built into the learner.

Two averages of the same loss

Given $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$, empirical risk minimisation picks $\hat{\theta} \in \arg \min_{\theta} \hat{R}_{\text{train}}(\theta)$. This solves a *fitting* problem. The purpose of learning is to perform well on new cases drawn from the environment in which the model will be used.

Same loss, two populations of examples: the ones you have, and the ones you will meet.

Definition 3.1.1 (Empirical risk, population risk, generalization gap). For a model f , the *empirical risk* on a dataset $\mathcal{D}$ and the *population risk* are

$$\hat{R}_{\mathcal{D}}(f) = \frac{1}{N} \sum_{i=1}^N \ell(y^{(i)}, f(x^{(i)})), \quad R(f) = \mathbb{E}_{(X,Y) \sim \mathbb{P}}[\ell(Y, f(X))].$$

For the model $\hat{f}_{\mathcal{D}}$ fitted on $\mathcal{D}$, the *generalization gap* is

$$G(\hat{f}_{\mathcal{D}}) = R(\hat{f}_{\mathcal{D}}) - \hat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}}).$$

Reading the formula

Both terms of the gap refer to the *same* fitted model; only the source of the examples differs. $\hat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}})$ is the score on the very examples that chose the model. $R(\hat{f}_{\mathcal{D}})$ is the score on fresh cases from the world. A gap near 0 means training performance transfers; a large positive gap means the training score was flattering. If a spam filter makes 2% errors on its training emails and 9% on the emails of the next month, the gap is 0.07.

Why training performance is special

Training data play two roles when training performance is reported: they first influence *which* model is selected, and are then used again to *evaluate* that model,

$$\mathcal{D} \longrightarrow \hat{f}_{\mathcal{D}} \longrightarrow \hat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}}).$$

The procedure can therefore exploit accidental regularities in $\mathcal{D}$. A second sample $\mathcal{D}'$ from the same population would generally produce another fitted model $\hat{f}_{\mathcal{D}'}$; this sampling variability is part of the learning problem, not a defect of the notation.

Proposition 3.1.2 (Training risk of a fitted model is optimistic). *Let the learning procedure return $\hat{f}_{\mathcal{D}}$ minimising $\hat{R}_{\mathcal{D}}$ over a class $\mathcal{H}$, with $\mathcal{D}$ an i.i.d. sample. Then*

$$\mathbb{E}_{\mathcal{D}}[\hat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}})] \leq \min_{f \in \mathcal{H}} R(f) \leq \mathbb{E}_{\mathcal{D}}[R(\hat{f}_{\mathcal{D}})],$$

so the expected generalization gap is nonnegative: $\mathbb{E}_{\mathcal{D}}[G(\hat{f}_{\mathcal{D}})] \geq 0$.

Proof. This is Proposition 2.5.3 of Chapter 2, restated. Let $f_{\mathcal{H}}^*$ minimise R over $\mathcal{H}$. Since $\hat{f}_{\mathcal{D}}$ minimises the empirical risk, $\hat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}}) \leq \hat{R}_{\mathcal{D}}(f_{\mathcal{H}}^*)$ for every sample; taking expectations and using that $f_{\mathcal{H}}^*$ is fixed (Proposition 2.5.2), the right side has expectation $R(f_{\mathcal{H}}^*)$. The second inequality holds pointwise because $R(\hat{f}_{\mathcal{D}}) \geq \min_{\mathcal{H}} R$. Subtracting gives the sign of the expected gap. $\square$

Interpretation & discussion

Low training loss is *necessary* evidence that optimisation has fitted the data, but *incomplete* evidence that learning has succeeded. Independent evaluation is needed to determine whether the fitted pattern persists beyond the observations that selected it. That is the entire justification for validation and test sets.

Stable and spurious regularities

Example 3.1.3 (Cows on grass, camels on sand). Suppose cows in the training collection are photographed on grass and camels on sand. *Both* animal shape and background predict every training label. Shape is closely related to the intended task; background is a shortcut created by the way the images were collected. Training performance cannot tell the two cues apart, because both work on every observed image. A cow photographed on sand separates them: the shape cue says “cow,” the background cue says “camel.” This is the wolves-and-snow story of Chapter 1 (Example 1.7.4) seen from the fitting side: a spurious cue produces zero training loss and a large generalization gap.

Dataset composition is therefore *part of the learning method*: the examples determine which correlations the model can exploit. Similar shortcuts occur when a network segment or capture time predicts attacks, a hospital or scanner type identifies a diagnosis, or a customer identifier predicts an outcome in historical records. These variables may be predictive in the sample without describing a relationship that remains useful at deployment.

Technique: Weakening shortcuts

- Vary nuisance factors at *collection* time: several cameras, sites, seasons, devices.
- Use *data augmentation* only with transformations that are realistic and preserve the label (flip a cat photo, yes; flip a digit “6”, no: it becomes a “9”).
- Evaluate under deployment-like conditions (Section 3.6); a random split hides shortcuts that are present in every part of the collection.

Inductive bias

Definition 3.1.4 (Inductive bias). A finite sample is compatible with many functions, including functions that agree on the sample and differ everywhere else (Proposition 1.7.2). Learning therefore requires an *inductive bias*: assumptions that make some solutions more plausible than others beyond the observed examples. It enters through

- the *data and representation*, which determine which cues are available;
- the *hypothesis space*, which determines which relationships can be expressed;
- *regularisation*, which favours some fitting solutions over others;
- the *optimisation algorithm*, which may prefer particular solutions when many parameters minimise the training objective.

Here “bias” means a preference among solutions; it is broader than the statistical bias of Section 3.3.

Key insight

Generalization is never a property of the data alone. It succeeds when the assumptions encoded by the learning procedure remain true in the deployment environment. Every technique in this chapter is a way of choosing, strengthening or checking those assumptions.

Summary: Generalization and inductive bias

- Empirical risk is observed but sample-dependent; population risk is wanted but unknown; the gap compares them for one fitted model.
- Training risk after fitting is optimistic in expectation (Proposition 3.1.2), because the data choose and then grade the model.
- Spurious cues fit the sample as well as stable ones; only new contexts separate them.
- Inductive bias (data, representation, hypothesis space, regularisation, optimiser) decides behaviour off the sample.

3.2 Model Capacity

What are we trying to get?

We want a vocabulary for “how flexible is this family of models,” a proof that flexibility always looks good on training data, and a concrete demonstration that it does not always look good on new data.

Definition 3.2.1 (Capacity, nested families). The *capacity* of a hypothesis space $\mathcal{H} = \{f_\theta : \theta \in \Theta\}$ describes how varied the functions in it can be. It is not determined by parameter count alone: structure, parameter constraints, regularisation, optimisation, and the amount and geometry of the data all matter. Typical capacity-controlling hyperparameters are polynomial degree, tree depth, the number of units in a network, and the neighbourhood size k in kNN. Families are *nested* when $\mathcal{H}_1 \subseteq \mathcal{H}_2 \subseteq \dots \subseteq \mathcal{H}_M$: every function of a smaller family remains available in a larger one.

Proposition 3.2.2 (Training error favours capacity). *For nested families $\mathcal{H}_m \subseteq \mathcal{H}_{m+1}$,*

$$\min_{f \in \mathcal{H}_{m+1}} \hat{R}_{\text{train}}(f) \leq \min_{f \in \mathcal{H}_m} \hat{R}_{\text{train}}(f).$$

Increasing capacity cannot make the best achievable training error worse.

Proof. A larger family can always reuse the best solution of a smaller one: the minimiser over $\mathcal{H}_m$ is a candidate in $\mathcal{H}_{m+1}$, so the minimum over $\mathcal{H}_{m+1}$ is at most its value. (Proposition 2.5.4 of Chapter 2.) $\square$

Reading the formula

This is why training loss *alone* systematically favours richer families: it can only go down as you add flexibility. It does not follow that the population risk goes down. Additional functions can fit stable structure, but they can also fit accidental variation.

Underfitting and overfitting

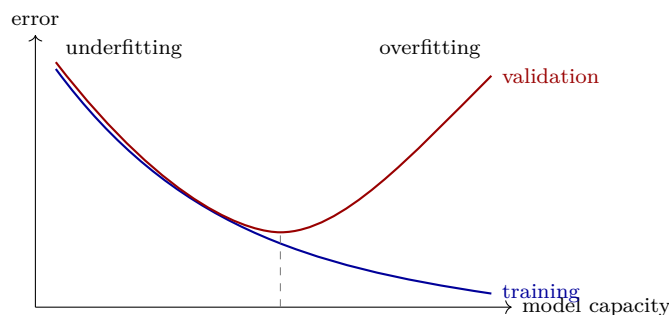
Definition 3.2.3 (Underfitting, overfitting). An *underfitting* model is too restrictive to capture important structure: it usually has high training *and* high validation error. An *overfitting* model is too sensitive to the particular training sample: it may have very low training error and substantially higher validation error. The distinction is not whether the model is complicated in isolation, but whether its effective flexibility matches the available evidence.

Example 3.2.4 (Three polynomials on four points, judged on a fifth). Chapter 2 (Example 2.5.6) fitted the points $(0, 1)$, $(1, 3)$, $(2, 2)$, $(3, 4)$ with polynomials of degree 0, 1 and 3, with training risks 1.25, 0.45 and 0. Now a *validation* point arrives: $(4, 5)$.

- Degree 0: the constant 2.5 predicts 2.5; squared error $(5 - 2.5)^2 = 6.25$.
- Degree 1: the line $0.8x + 1.3$ predicts 4.5; squared error 0.25.
- Degree 3: the interpolating cubic is $f(x) = 1 + 5.5x - 4.5x^2 + x^3$ (check: $f(1) = 3$, $f(2) = 2$, $f(3) = 4$); it predicts $f(4) = 1 + 22 - 72 + 64 = 15$; squared error $(5 - 15)^2 = 100$.

degree	0	1	3
training risk	1.25	0.45	0.00
validation error at $x = 4$	6.25	0.25	100

The constant underfits (bad everywhere). The cubic overfits: perfect on the four points it was shaped by, and wildly wrong one step outside them, because it bent itself around the noise in the sample. The line, which fits the training data worst of the last two, is the only one worth deploying.



For kNN the direction of the scale is reversed: small k gives small, highly adaptive neighbourhoods (low training error, high sensitivity), large k averages broadly (smoother rule, higher training error). Capacity must also be read relative to sample size: a family that is unstable with a small dataset may be well con-

strained by a much larger one, because more representative observations restrict the functions that can fit the sample while behaving arbitrarily elsewhere.

Gotcha

- “Zero training error” is not an achievement; with enough capacity it is guaranteed (Proposition 2.5.5). It is a warning to look at validation error.
- For kNN, *decreasing* k increases capacity. Students routinely get this backwards.

Summary: Model capacity

- Capacity: how varied the functions in $\mathcal{H}$ can be; controlled by degree, depth, width, k , and more.
- Nested families: training error never increases with capacity (Proposition 3.2.2); validation error typically falls then rises.
- Underfitting: high train and validation error. Overfitting: low train, high validation.

3.3 Bias and Variance

What are we trying to get?

“Sensitive to the sample” is a vague phrase. We want to make it a number, separate it from a second kind of error (being systematically wrong), and prove that squared prediction error splits exactly into noise plus these two quantities. That decomposition is the theoretical spine of the whole chapter.

A repeated-sampling thought experiment

Imagine repeatedly drawing training sets $\mathcal{D}_1, \mathcal{D}_2, \dots$ from the same population and applying exactly the same learning procedure. The fitted functions $\hat{f}_{\mathcal{D}_1}, \hat{f}_{\mathcal{D}_2}, \dots$ need not coincide.

Imagine rerunning the entire training procedure on many different samples from the same world, then looking at the cloud of predictions you would get at one input.

Definition 3.3.1 (Bias and variance of a learning procedure). For squared-error regression let $f^*(x) = \mathbb{E}[Y \mid X = x]$ be the optimal regression function and $\bar{f}(x) = \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)]$ the average prediction of the procedure at x . The *pointwise bias* and *pointwise variance* are

$$\text{Bias}(x) = \bar{f}(x) - f^*(x), \quad \text{Var}(x) = \mathbb{E}_{\mathcal{D}}[(\hat{f}_{\mathcal{D}}(x) - \bar{f}(x))^2].$$

Reading the formula

Fix one input x , say a particular house. Train a thousand times on a thousand different training sets and record the thousand predicted prices. $\bar{f}(x)$ is their average. *Bias* asks: is that average off from the true conditional mean price, in a consistent direction? *Variance* asks: how widely do the thousand predictions scatter around their own average? A model that always predicts “1 million” has zero variance and possibly large bias; a very flexible model may have tiny

bias and huge variance.

Restrictive models and inadequate features create bias; merely collecting more examples may not remove it if the family cannot express the relationship. High capacity, small datasets, irrelevant variables and unstable feature selection increase variance; more representative data reduce it.

Theorem 3.3.2 (Bias–variance decomposition). *Assume $Y = f^*(X) + \varepsilon$ with $\mathbb{E}[\varepsilon | X] = 0$ and $\text{Var}(\varepsilon | X = x) = \sigma^2(x)$, and assume the training sample $\mathcal{D}$ is independent of the test pair (x, Y) . Then at every fixed input x ,*

$$\mathbb{E}_{\mathcal{D}, Y|x}[(Y - \hat{f}_{\mathcal{D}}(x))^2] = \sigma^2(x) + \text{Bias}(x)^2 + \text{Var}(x).$$

Proof. **Step 1 (split off the noise).** Write $Y - \hat{f}_{\mathcal{D}}(x) = \varepsilon + (f^*(x) - \hat{f}_{\mathcal{D}}(x))$, where $\varepsilon = Y - f^*(x)$. Squaring,

$$(Y - \hat{f}_{\mathcal{D}}(x))^2 = \varepsilon^2 + 2\varepsilon(f^*(x) - \hat{f}_{\mathcal{D}}(x)) + (f^*(x) - \hat{f}_{\mathcal{D}}(x))^2.$$

Take expectations. $\mathbb{E}[\varepsilon^2 | x] = \sigma^2(x)$. In the cross term, ε is independent of $\mathcal{D}$ (the test noise has nothing to do with the training sample) and has conditional mean 0, so $\mathbb{E}[\varepsilon(f^*(x) - \hat{f}_{\mathcal{D}}(x))] = \mathbb{E}[\varepsilon | x] \cdot \mathbb{E}_{\mathcal{D}}[f^*(x) - \hat{f}_{\mathcal{D}}(x)] = 0$.

Step 2 (split the remaining error around the average prediction). Write $f^*(x) - \hat{f}_{\mathcal{D}}(x) = (f^*(x) - \bar{f}(x)) + (\bar{f}(x) - \hat{f}_{\mathcal{D}}(x))$. Squaring and taking $\mathbb{E}_{\mathcal{D}}$: the first bracket is a constant, $-\text{Bias}(x)$, whose square is $\text{Bias}(x)^2$; the second has $\mathbb{E}_{\mathcal{D}}$ -mean zero by definition of $\bar{f}$, so the cross term $2(f^*(x) - \bar{f}(x)) \mathbb{E}_{\mathcal{D}}[\bar{f}(x) - \hat{f}_{\mathcal{D}}(x)]$ vanishes; and the square of the second bracket averages to $\text{Var}(x)$.

Adding the pieces: $\sigma^2(x) + \text{Bias}(x)^2 + \text{Var}(x)$. $\square$

Key insight

Twice the same move as in Theorem 2.2.4: add and subtract a mean, square, and watch the cross term die because a mean-zero quantity multiplies a constant. First around the true regression function (killing the noise cross term), then around the average prediction (killing the sampling cross term).

Reading the formula

Three sources of error at one input.

- $\sigma^2(x)$: *irreducible noise*. Even the perfect predictor f^* makes this error, because Y has randomness that x does not explain.
- $\text{Bias}(x)^2$: *systematic error* of the procedure, the part that repeating the experiment never averages away.
- $\text{Var}(x)$: *sample sensitivity*, the part that a different training set would change.

Increasing capacity often reduces bias while increasing variance, producing the U-shaped validation curve. This is a useful diagnostic principle, not a universal law.

Example 3.3.3 (Bias and variance computed exactly). The simplest regression problem: no input, $Y = \mu + \varepsilon$ with $\text{Var}(\varepsilon) = \sigma^2$, so $f^* = \mu$. The training set is N draws $y^{(1)}, \dots, y^{(N)}$. Three procedures:

- *Sample mean*, $\hat{f} = \frac{1}{N} \sum_i y^{(i)}$. $\mathbb{E}_{\mathcal{D}}[\hat{f}] = \mu$: bias 0. Variance of a mean of N independent draws: σ^2/N . Total error $\sigma^2 + 0 + \sigma^2/N$.
- *Ignore the data*, $\hat{f} = 0$. Bias $-\mu$, variance 0. Total $\sigma^2 + \mu^2$.
- *Shrunk mean*, $\hat{f} = \frac{1}{2} \cdot \frac{1}{N} \sum_i y^{(i)}$. $\mathbb{E}_{\mathcal{D}}[\hat{f}] = \mu/2$: bias $-\mu/2$. Variance $\frac{1}{4}\sigma^2/N$. Total $\sigma^2 + \mu^2/4 + \sigma^2/(4N)$.

With $\sigma^2 = 1$ and $N = 4$:

bias ² + variance	sample mean	ignore data	shrunk mean
$\mu = 1$	$0 + 0.25 = 0.25$	$1 + 0 = 1$	$0.25 + 0.0625 = 0.3125$
$\mu = 0.2$	$0 + 0.25 = 0.25$	$0.04 + 0 = 0.04$	$0.01 + 0.0625 = 0.0725$

When the truth is far from zero ($\mu = 1$) the unbiased procedure wins. When the truth is near zero ($\mu = 0.2$) both biased procedures beat it: they trade a little systematic error for a large reduction in variance. Regularisation (Section 3.4) is exactly this trade, made tunable.

Beyond the classical U-shape

Remark 3.3.4 (Double descent). Modern overparameterised models can display *double descent*. As capacity approaches the smallest level at which the training sample can be interpolated, validation error may increase sharply; beyond this *interpolation threshold*, further capacity can reduce validation error again. The second descent depends on regularisation, the optimiser, the data geometry and which interpolating solution is selected; it does not occur in every problem. An analogous phenomenon can occur over *training time* with a fixed number of parameters. The classical U-shape remains a valuable first model, but validation behaviour in modern systems must be examined rather than assumed.

Definition 3.3.5 (Three sources of excess error). Relative to the Bayes risk, a learned model's extra risk decomposes conceptually into

- *approximation error*: the hypothesis space cannot express a good enough function;
- *estimation error*: finite data do not reliably identify the best function in the space;
- *optimisation error*: the algorithm fails to reach a good solution of its own objective.

Increasing capacity can reduce approximation error while increasing estimation error. Optimisation is studied later in the course.

Classic problems

- A procedure has $\text{Bias}(x) = 0.3$, $\text{Var}(x) = 0.05$, noise $\sigma^2 = 0.1$. Expected squared error at x ? $0.1 + 0.09 + 0.05 = 0.24$.
- Doubling the data leaves bias unchanged and halves the variance in Example 3.3.3. True in general? The variance part often shrinks with N ; the bias does not, because it comes from the family, not the sample size.
- Can a procedure have zero bias and zero variance? Only if it returns f^* regardless of the sample, i.e. if it already knows the answer.

- 1-NN regression: *bias and variance?* Bias small (it follows the data closely), variance large (each prediction depends on one noisy neighbour).

Summary: Bias and variance

- Bias: average prediction minus truth, across training sets. Variance: spread of predictions across training sets.
- Squared error = noise + bias² + variance (Theorem 3.3.2), by adding and subtracting means twice.
- Capacity usually trades bias for variance; a little bias can buy a lot of variance (Example 3.3.3); double descent shows the trade is not the whole story.

3.4 Controlling Complexity

What are we trying to get?

If flexibility is the problem, we need dials that reduce it *without* throwing away parameters: penalties on parameter size, and stopping training early. We want to see exactly what a penalty does to the hypothesis space, why the L_1 penalty produces exact zeros, and how the strength of the penalty is chosen.

Complexity can be controlled by restricting the family (degree, depth), increasing k in kNN, selecting features, penalising parameters, stopping training early, or collecting more relevant data. Each choice expresses an assumption about which fitted solutions are likely to generalise.

Regularised empirical risk minimisation

Definition 3.4.1 (Regularised ERM). Given a *penalty* $\Omega : \Theta \rightarrow [0, \infty)$ and a strength $\lambda \geq 0$,

$$\hat{\theta}_\lambda \in \arg \min_{\theta \in \Theta} [\hat{R}_{\text{train}}(\theta) + \lambda \Omega(\theta)].$$

Two standard penalties are

$$\Omega_2(\theta) = \|\theta\|_2^2 = \sum_j \theta_j^2 \quad (L_2, \text{ ridge}), \quad \Omega_1(\theta) = \|\theta\|_1 = \sum_j |\theta_j| \quad (L_1, \text{ lasso}).$$

Reading the formula

The objective is now “fit the data *and* keep the parameters small.” λ says how much a unit of penalty is worth in units of training loss: $\lambda = 0$ is plain ERM; large λ makes any sizeable coefficient too expensive, so the fit is pulled toward the origin. With two coefficients $(3, -4)$, $\Omega_2 = 9 + 16 = 25$ and $\Omega_1 = 3 + 4 = 7$.

Theorem 3.4.2 (A penalty is a soft constraint). Let $\hat{\theta}_\lambda$ solve the penalised

problem and put $c = \Omega(\hat{\theta}_\lambda)$. Then $\hat{\theta}_\lambda$ also solves the constrained problem

$$\min_{\theta} \hat{R}_{\text{train}}(\theta) \quad \text{subject to} \quad \Omega(\theta) \leq c.$$

Consequently regularised ERM fits within the family $\mathcal{H}_c = \{f_\theta : \Omega(\theta) \leq c\}$, and these families are nested: $c_1 < c_2 \Rightarrow \mathcal{H}_{c_1} \subseteq \mathcal{H}_{c_2}$.

Proof. Take any θ with $\Omega(\theta) \leq c$. Optimality of $\hat{\theta}_\lambda$ for the penalised problem gives $\hat{R}(\hat{\theta}_\lambda) + \lambda c \leq \hat{R}(\theta) + \lambda \Omega(\theta) \leq \hat{R}(\theta) + \lambda c$, hence $\hat{R}(\hat{\theta}_\lambda) \leq \hat{R}(\theta)$: no feasible θ beats $\hat{\theta}_\lambda$, and $\hat{\theta}_\lambda$ is itself feasible. Nesting is immediate from $\Omega(\theta) \leq c_1 \Rightarrow \Omega(\theta) \leq c_2$. $\square$

Key insight

A stronger penalty confines the solution to a smaller *effective* family, even though the number of parameters has not changed. Capacity is about which functions can actually win the fitting contest, not about how many numbers the model stores. The precise map from λ to c depends on the problem.

Proposition 3.4.3 (Ridge in one variable: shrinkage). *For the model $f_w(x) = wx$ (no intercept) under squared loss with L_2 penalty, the regularised solution is*

$$\hat{w}_\lambda = \frac{\sum_i x^{(i)} y^{(i)}}{\sum_i (x^{(i)})^2 + N\lambda},$$

which is the unpenalised least-squares slope multiplied by the factor $\frac{\sum_i (x^{(i)})^2}{\sum_i (x^{(i)})^2 + N\lambda} \in (0, 1]$: the coefficient is shrunk toward 0, smoothly, never exactly to 0 for finite λ .

Proof. The objective $J(w) = \frac{1}{N} \sum_i (y^{(i)} - wx^{(i)})^2 + \lambda w^2$ is a convex quadratic in w . Setting $J'(w) = -\frac{2}{N} \sum_i x^{(i)}(y^{(i)} - wx^{(i)}) + 2\lambda w = 0$ gives $w(\sum_i (x^{(i)})^2 + N\lambda) = \sum_i x^{(i)} y^{(i)}$. The factor form follows by dividing numerator and denominator by $\sum_i (x^{(i)})^2$. $\square$

Reading the formula

With $\sum x^2 = 14$, $\sum xy = 19$ and $N = 4$ (the numbers of Example 2.5.6, ignoring the intercept): $\lambda = 0$ gives $19/14 \approx 1.357$; $\lambda = 1$ gives $19/18 \approx 1.056$; $\lambda = 10$ gives $19/54 \approx 0.352$. The bigger the penalty, the closer to zero, but never zero.

Proposition 3.4.4 (Lasso in one variable: soft thresholding). *For $z \in \mathbb{R}$ and $\lambda > 0$, the minimiser of $\frac{1}{2}(w - z)^2 + \lambda|w|$ over $w \in \mathbb{R}$ is*

$$\hat{w} = \text{sign}(z) \max(|z| - \lambda, 0).$$

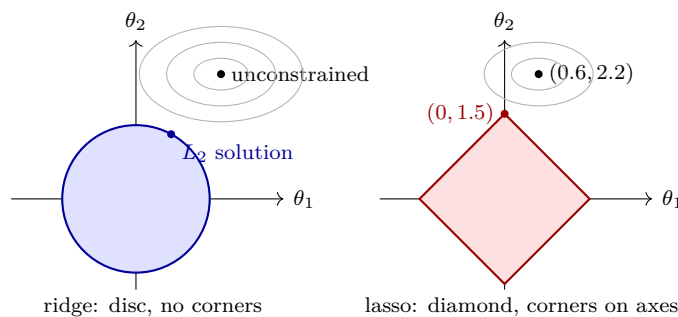
In particular $\hat{w} = 0$ exactly whenever $|z| \leq \lambda$.

Proof. Suppose first $z > 0$. For $w \geq 0$ the objective is $\frac{1}{2}(w - z)^2 + \lambda w$, a convex parabola with unconstrained minimiser $w = z - \lambda$; if $z > \lambda$ this is positive and gives the minimum on $[0, \infty)$; if $z \leq \lambda$ the parabola is increasing on $[0, \infty)$ and the minimum on that side is at $w = 0$. For $w < 0$ the objective is $\frac{1}{2}(w - z)^2 - \lambda w$,

which is decreasing in w toward 0 when $z > 0$ (its unconstrained minimiser $z + \lambda$ is positive), so no negative w beats $w = 0$. Combining: $\hat{w} = \max(z - \lambda, 0)$ for $z > 0$. The case $z < 0$ is symmetric, and $z = 0$ gives $\hat{w} = 0$. $\square$

Reading the formula

z is where the unpenalised fit would put the coefficient. The L_1 penalty moves it toward zero by exactly λ , and if it would cross zero, it stops *at* zero. With $\lambda = 0.5$: $z = 2.2 \rightarrow 1.7$; $z = 0.4 \rightarrow 0$; $z = -3 \rightarrow -2.5$. Geometrically, the constraint region $\|\theta\|_1 \leq c$ is a diamond whose corners lie on the coordinate axes, and an optimum often lands on a corner where some coordinates are exactly zero. The slide's two-dimensional example has unconstrained optimum $(0.6, 2.2)$ and lasso solution $(0, 1.5)$: the first feature is dropped from the model altogether. L_1 therefore performs *embedded feature selection*, although correlated features can make the selected subset unstable. L_2 's region is a disc with no corners, so it shrinks everything and keeps everything.



Choosing the strength, and early stopping

The regularisation strength is a hyperparameter: for each candidate λ , training estimates $\hat{\theta}_\lambda$; validation selects $\hat{\lambda} \in \arg \min_{\lambda \in C} \hat{R}_{\text{val}}(\hat{f}_\lambda)$; the test data stay outside this selection (Section 2.6).

Definition 3.4.5 (Early stopping). Iterative optimisation produces a sequence $\hat{\theta}_0, \hat{\theta}_1, \dots$. Early iterations often learn strong recurring patterns; later ones may fit exceptional observations or noise. *Early stopping* retains the checkpoint with smallest validation risk,

$$t^* = \arg \min_t \hat{R}_{\text{val}}(\hat{\theta}_t), \quad \hat{\theta}_{\text{ES}} = \hat{\theta}_{t^*}.$$

Training time thereby controls effective capacity while the parameter count stays fixed. A *patience* p lets training continue p epochs without improvement before stopping, keeping the best checkpoint seen.

Reading the formula

Every epoch, compute the validation loss; remember the epoch where it was lowest; when you finally stop, roll back to that epoch's parameters. Patience protects against stopping on a temporary wobble of a noisy validation curve. It can still miss a later decrease under epoch-wise double descent; a longer horizon with checkpoint retention is safer when that is plausible, at the price of computation, and with no guarantee that a second descent will come.

Technique: Regularisation in practice

- Standardise features before penalising (Section 2.6); otherwise a feature measured in millimetres is penalised far more than the same feature in metres.
- Do not penalise the intercept.
- Search λ on a logarithmic grid ($10^{-3}, 10^{-2}, \dots, 10^2$); the useful range spans orders of magnitude.
- Use L_1 when you believe few features matter and want to know which; L_2 when many features contribute a little.
- Early stopping is regularisation for free in any iterative method; always keep the best checkpoint, not the last.

Classic problems

- *Soft-threshold* $z = 1.2$ with $\lambda = 1.5$. $|z| \leq \lambda$: $\hat{w} = 0$.
- *Ridge* with $\lambda \rightarrow \infty$? All penalised coefficients $\rightarrow 0$; the model becomes the intercept alone (degree-0 fit).
- *Two identical features under L_2 versus L_1* . L_2 splits the weight equally between them; L_1 is indifferent among all splits and may pick either, which is the instability mentioned above.
- *Why is λ chosen on validation and not on training data?* Training risk is minimised at $\lambda = 0$ (Proposition 3.2.2 applied to the nested families $\mathcal{H}_c$).

Summary: Controlling complexity

- Regularised ERM: minimise training risk plus $\lambda\Omega(\theta)$; equivalent to fitting within a nested constrained family (Theorem 3.4.2).
- L_2 shrinks smoothly and keeps all features (Proposition 3.4.3); L_1 soft-thresholds and sets coefficients exactly to zero (Proposition 3.4.4).
- λ and the stopping epoch are hyperparameters chosen on validation data.

3.5 Feature and Model Selection

What are we trying to get?

Choosing *which inputs* to use is a model choice like any other, and it can go wrong in the same ways. We want to see why exhaustive search is impossible, what the practical searches do, why they must live inside the training pipeline, and why the validation set itself can be worn out by overuse.

Feature selection changes the representation supplied to the learner. It can remove irrelevant variables that increase estimation difficulty, reduce redundancy, improve interpretability, and exclude variables unavailable at decision time. It is a data-dependent model choice and must be evaluated accordingly.

Proposition 3.5.1 (Subset selection is combinatorial). *With d features there are 2^d subsets. The idealised search $S^* \in \arg \min_{S \subseteq \{1, \dots, d\}} \widehat{R}_{\text{val}}(\widehat{f}_S)$, where $\widehat{f}_S$ uses only the features in S , requires 2^d fits. For $d = 30$, $2^{30} = 1,073,741,824$.*

Proof. Each feature is independently in or out of the subset: $2 \times \dots \times 2 = 2^d$ choices. $\square$

Definition 3.5.2 (Filters and greedy searches). • A *correlation filter* scores each variable separately, e.g. $s_j = |\widehat{\text{Corr}}(X_j, Y)|$, and keeps the top-scoring ones. Fast and model-independent, but it misses nonlinear relationships and interactions, and it ignores redundancy: two highly ranked features may carry the same information. Weak marginal correlation does not prove that a feature is useless jointly with others.

- *Forward selection* starts from $S = \emptyset$ and repeatedly adds the feature giving the largest validation improvement. *Backward elimination* starts from all features and repeatedly removes the least useful. Both are greedy: an early choice can block a better combination later; backward elimination also requires fitting the full model.

Example 3.5.3 (Two features that are useless alone and perfect together). Let X_1, X_2 be independent fair coin flips (± 1) and $Y = X_1 X_2$. Each feature has correlation 0 with Y (knowing X_1 alone says nothing about the sign of the product), so a correlation filter discards both. Together they determine Y exactly. Filters see one variable at a time; interactions are invisible to them.

Gotcha

All data-dependent feature scoring and subset search belong *inside* the training pipeline. In cross-validation they must be repeated separately within every training fold. Otherwise information from a validation fold shapes the representation that is then evaluated on that fold: *leakage*, and an optimistic estimate.

The roles of the three subsets, and their overuse

Let h denote an entire hyperparameter configuration (features, model family, λ , k , thresholds), not only one number. The development loop fits $\widehat{f}_h$ on training data, measures validation performance, and updates the configuration: $\hat{h} \in \arg \min_{h \in \mathcal{C}} \widehat{R}_{\text{val}}(\widehat{f}_h)$. The test set remains outside this loop.

Proposition 3.5.4 (The best of many validation scores is optimistic). *Let $\widehat{R}_{\text{val}}(\widehat{f}_h)$, $h \in \mathcal{C}$, be validation risks of models fitted without the validation data, so that each is an unbiased estimate of $R(\widehat{f}_h)$ (Proposition 2.6.2). Then*

$$\mathbb{E} \left[\min_{h \in \mathcal{C}} \widehat{R}_{\text{val}}(\widehat{f}_h) \right] \leq \min_{h \in \mathcal{C}} \mathbb{E} [\widehat{R}_{\text{val}}(\widehat{f}_h)] = \min_{h \in \mathcal{C}} R(\widehat{f}_h),$$

with strict inequality as soon as the estimates are noisy and no single h is best in every sample.

Proof. For every $h_0 \in \mathcal{C}$, $\min_h \widehat{R}_{\text{val}}(\widehat{f}_h) \leq \widehat{R}_{\text{val}}(\widehat{f}_{h_0})$ pointwise; take expectations, then take the minimum over h_0 of the right side. $\square$

Reading the formula

Try a hundred configurations and report the lowest validation score: that number is the minimum of a hundred noisy estimates, and minima of noisy things are pulled downward. Every reported validation result also influences the next manual or automated design choice, so the development process gradually adapts to the validation set. The final test set protects against this accumulated adaptation; it does *not* repair it if its results are then used to continue development, because at that point it has become another validation set.

Definition 3.5.5 (K -fold cross-validation, restated for configurations). Partition the development data into K folds. For configuration h and fold j , fit $\hat{f}_h^{(-j)}$ without fold j and evaluate it on fold j ; then

$$\hat{R}_{\text{CV}}(h) = \frac{1}{K} \sum_{j=1}^K \hat{R}_{\text{val}}^{(j)}(\hat{f}_h^{(-j)}), \quad \hat{h} \in \arg \min_{h \in \mathcal{C}} \hat{R}_{\text{CV}}(h).$$

Each observation validates once and trains $K - 1$ times.

Cross-validation uses limited development data efficiently, at roughly K fits per configuration; its estimate still varies with the available sample; standard random folds assume exchangeable observations, so grouped or temporal dependence needs adapted folds (Section 3.6); and it does not replace final evaluation on an untouched test set.

Summary: Feature and model selection

- 2^d subsets make exhaustive selection impossible; filters and greedy searches are the practical tools, each blind to something.
- Selection is part of training: redo it inside every fold, or leak.
- The minimum of many validation scores is optimistic (Proposition 3.5.4); hence a test set used once.

3.6 Reliable Evaluation and Data Splitting

What are we trying to get?

An evaluation is only as honest as its split. We want a rule for choosing the split from the deployment question, metrics that do not hide minority-class failure, and a checklist of the ways information leaks from the future into the model.

The split should match deployment

If the model must predict future events, the evaluation should preserve time order. If it must generalise to new users, devices, patients or organisations, entire groups should be held out. A random split is credible only when new deployment cases are reasonably approximated by independent draws from the same population.

*Ask first: what kind of new case will the model face?
Then hold out exactly that kind of case.*

Definition 3.6.1 (Random, stratified, grouped and temporal splits). •

Random: each observation is assigned to a subset approximately independently.

- *Stratified*: random, but preserving approximate class proportions in every subset, which stabilises rare-class counts. Stratification must not override dependence: records from one device should not be scattered across subsets merely to match proportions.
- *Grouped*: every group (user, device, patient, network) lies entirely in one subset, so the learner cannot use identity-specific patterns and is evaluated on unseen groups.
- *Temporal*: train on the past, validate and test on later observations; this preserves the direction of prediction and reveals deterioration as behaviour, sensors or attack patterns change.

Example 3.6.2 (The same intrusion dataset, three verdicts). Flows are recorded from 50 devices over 6 months. A *random* split puts flows from every device and every week into train and test; the model can learn “device 17 is the compromised laptop” and score 99.5%. A *grouped* split holds out 10 devices; the model must recognise attack *behaviour* and scores 96%. A *temporal* split trains on months 1 to 4 and tests on months 5 and 6, during which a new attack tool appeared; the model scores 90%. Only the third number describes what will happen next month. The first is not wrong arithmetic; it answers a question nobody asked.

Class imbalance and metrics

Proposition 3.6.3 (Accuracy hides minority failure). *If a fraction π of the data is positive, the classifier that always predicts “negative” has accuracy $1 - \pi$ and recall 0 on the positive class.*

Proof. It is correct on every negative and wrong on every positive: accuracy = $(1 - \pi)$; it detects no positive, so $TP = 0$ and recall = $0/(0 + FN) = 0$. $\square$

With $\pi = 0.01$ this is 99% accuracy and no attack ever detected.

Definition 3.6.4 (Metrics from the confusion matrix). With attacks as the positive class, count true positives TP, false negatives FN, false positives FP and true negatives TN on the evaluation set. Then

$$\text{Recall} = \frac{TP}{TP + FN}, \quad \text{Precision} = \frac{TP}{TP + FP}, \quad \text{Specificity} = \frac{TN}{TN + FP},$$

$$\text{BalancedAcc} = \frac{\text{Recall} + \text{Specificity}}{2}, \quad F_1 = 2 \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Reading the formula

Each metric answers one question. *Recall*: of the real attacks, what fraction did we catch? *Precision*: of the alarms we raised, what fraction were real? *Specificity*: of the benign flows, what fraction did we leave alone? *Balanced*

accuracy: average of the two per-class rates, immune to imbalance. F_1 : the harmonic mean of precision and recall, high only when both are high; it ignores true negatives and treats the two symmetrically.

A detector on 10,000 flows with 50 attacks: TP = 40, FN = 10, FP = 90, TN = 9860.

metric	computation	value
accuracy	9900/10000	0.990
recall	40/50	0.80
precision	40/130	0.31
specificity	9860/9950	0.991
balanced accuracy	$(0.80 + 0.991)/2$	0.895
F_1	$2(0.31)(0.80)/(0.31 + 0.80)$	0.44

Accuracy says 99%; precision says two alarms in three are false. Which number matters depends on whether an analyst has to investigate each alarm.

Proposition 3.6.5 (F_1 lies between the smaller and the average of precision and recall). For $P, R \in (0, 1]$: $\min(P, R) \leq F_1 \leq \frac{P+R}{2}$, with $F_1 = \frac{P+R}{2}$ iff $P = R$.

Proof. $F_1 = 2PR/(P + R)$. Lower bound: if $P \leq R$ then $F_1 = P \cdot \frac{2R}{P+R} \geq P$ because $2R \geq P + R$. Upper bound: $\frac{P+R}{2} - \frac{2PR}{P+R} = \frac{(P+R)^2 - 4PR}{2(P+R)} = \frac{(P-R)^2}{2(P+R)} \geq 0$, zero iff $P = R$. $\square$

Interpretation & discussion

F_1 is dragged toward the *worse* of the two rates: precision 0.31 and recall 0.80 give $F_1 = 0.44$, far below their average 0.56. That is the point of a harmonic mean: it will not let a great recall hide a poor precision. No single metric is universally correct; metric and threshold choice should reflect the costs of missed attacks, false alarms and alert volume, exactly as in Theorem 2.2.9.

Data leakage

Definition 3.6.6 (Leakage). *Leakage* occurs when development uses information that would not be available when a new deployment decision is made. Common forms:

- duplicate or near-duplicate events in several subsets;
- records from the same device, session or patient in training and test data;
- predictors created after the outcome or after a human investigation (a “ticket closed as intrusion” field);
- preprocessing fitted using validation or test observations;
- hyperparameters repeatedly chosen using test results.

Leakage creates an unrealistically favourable estimate of generalization.

Technique: Preprocessing belongs inside the training pipeline

For standardisation, the training set estimates

$$\hat{\mu}_j = \frac{1}{N_{\text{tr}}} \sum_{i \in \text{tr}} x_j^{(i)}, \quad \hat{\sigma}_j^2 = \frac{1}{N_{\text{tr}}} \sum_{i \in \text{tr}} (x_j^{(i)} - \hat{\mu}_j)^2,$$

and the *same* $\hat{\mu}_j, \hat{\sigma}_j$ are reused unchanged on validation, test and deployment data. The same rule applies to imputation, feature selection, dimensionality reduction and resampling. In cross-validation each fold fits these operations on its own training portion only.

Leaking pipeline	Valid pipeline
all data $\rightarrow$ estimate $\mu_j, \sigma_j \rightarrow$ split	split $\rightarrow$ estimate μ_j, σ_j on training $\rightarrow$ reuse them on validation, test, deployment

Classic problems

- TP = 30, FN = 70, FP = 0. *Precision, recall, F_1 ?* 1.0, 0.3, $2(0.3)/(1.3) \approx 0.46$.
- *Which split for “will this customer churn next quarter”?* Temporal: features from before the quarter, outcome after.
- *Which split for “does this model work on a hospital it has never seen”?* Grouped by hospital.
- *A feature “number of analyst comments” predicts intrusions perfectly. Why is it useless?* Comments are written after the investigation: leakage from the future.

Summary: Reliable evaluation

- Choose the split from the deployment question: temporal for future events, grouped for new entities, random only for i.i.d. cases; stratify without breaking groups or time.
- Accuracy hides minority failure (Proposition 3.6.3); use recall, precision, specificity, balanced accuracy, F_1 as the costs dictate.
- Leakage in any form inflates the estimate; fit every preprocessing step on training data only.

3.7 Diagnosis and Lab Preparation

Definition 3.7.1 (Learning curves). *Learning curves* vary the amount of training data and plot training and validation performance against it.

Observed pattern	Likely issue	Possible response
Training and validation errors both high	underfitting	improve the representation or increase capacity
Training error low, validation error high	high variance	regularise, reduce capacity, or collect representative data
Errors close and both decrease as N grows	data-limited	additional representative examples are likely to help
Test performance degrades over time	distribution shift	update the data and use a time-aware evaluation protocol

“Errors close and decreasing” means the two curves have a small gap and both improve with more observations: the model is not overfitting strongly; the remaining limitation is the amount of evidence. Curves are diagnostic evidence, not proofs, and are read together with domain knowledge and error analysis.

Example 3.7.2 (Lab 1 at a glance). The intrusion-detection lab audits a labelled-flow dataset (imbalance, missing or non-finite values, duplicates, conflicting labels, redundancy), pre-declares random-row, grouped and chronological train/validation/test protocols, fits baselines (majority class, depth-3 tree, logistic regression, kNN) under each protocol, probes metadata shortcuts and shuffled labels, selects the decision threshold on validation, and reports test recall, false-positive rate and false alarms per 100,000 benign flows. Every idea of this chapter appears there: the protocols are Section 3.6, the shortcut probes are Section 3.1, the threshold-on-validation rule is Section 3.5, and the metrics are Definition 3.6.4.

Notes on the original slides and notes

All formulas were rederived and all counts checked ($2^{30} = 1,073,741,824$; the metric definitions; the bias–variance identity). Points to note:

- **Bias–variance decomposition (notes eq. 8, slide 24).** The notes write $\text{Var}(\varepsilon \mid X = x) = \sigma^2(x)$ while the slide writes a constant σ^2 ; both are fine, the proof never uses constancy. The identity also needs the training sample to be independent of the test noise, which the notes leave implicit; Theorem 3.3.2 states it.
- **“Moderate complexity: low bias, low variance” (slide 21).** This describes the sweet spot of the U-curve, not a general possibility; both terms are typically positive there, just small.
- **Typographic slips in the notes.** Section 4.2 has “widehat θ_1 ” (a rendering error for $\hat{\theta}_1$); Section 5 has “For ($\subseteq \{1, \dots, d\}$)” where the subset name S is missing; Section 1.2’s parenthesised symbols “(D), (P)” are formatting artefacts, not new notation.
- **Soft-constraint view (notes 4.1, slide 29).** The notes say the correspondence between λ and c “depends on the problem” without stating it; Theorem 3.4.2 gives the exact statement: $c = \Omega(\hat{\theta}_\lambda)$.
- **Lasso example (slide 31).** The numbers $(0.6, 2.2) \rightarrow (0, 1.5)$ are illustrative; with correlated features the lasso solution is not obtained by coordinatewise soft thresholding, which is why Proposition 3.4.4 is stated in one variable.

- **Slide 45.** “Always predicts normal” has recall 0 on attacks; its specificity is 1 and its balanced accuracy 0.5, which is the number that exposes it.
- **Slide 27.** “Optimization error will be developed in Lecture 6”: not covered here.

Chapter takeaways

Summary: From fitting to generalization

- **Low training loss does not establish performance on new data:** the sample chooses and then grades the model.
- **Inductive bias** enters through data, representation, model family, regularisation and optimisation; it decides what happens off the sample.
- **Capacity** trades approximation ability against sensitivity to a finite sample; training error only ever rewards more of it.
- **Bias** is systematic error across repeated samples, **variance** is variation across them; squared error = noise + bias² + variance.
- **Regularisation**, feature selection and early stopping control effective capacity; validation selects their strength; L_1 zeroes coefficients, L_2 shrinks them.
- **Evaluation splits** must reproduce the novelty expected at deployment; metrics must expose minority-class failure; **leakage** invalidates the estimate.